UMDCTF crypto/weave

Trapdoor Gabidulin unweaving

Author: AnyMoR Entry JOL

Reference basis: writeup_weave_v1.docx | Complements merged without redundancy from writeup_weave.docx and v7

Subject

Challenge statement

```
crypto/weave - ph1nx_
56 solves / 145 points

every piece of the cipher lies open on the table. compose them correctly and the flag is yours.

Downloads: chall.sage, output.json
```

Objective

Recover the AES-GCM key by composing the public weaving pieces correctly: rebuild GF(2^43), cancel the shuttle mask, decode a Gabidulin word with rank error t=15, untie knot, derive SHA256(secret_bytes)[:16], then verify and decrypt the AES-GCM vault.

Final flag: UMDCTF{l01dr34u_l4mbda3_brick5_th3_w34v3_but_th3_trapd00r_unsp00ls_1t}

License

1. Context and final result

The challenge gives chall.sage and output.json. The Sage generator shows that warp is not random: warp = knot * loom * shuttle^-1, and bolt = secret * warp + frays. Since knot, shuttle, pegs, fibers and the AES-GCM vault are all published, the real task is to put these pieces in the correct algebraic order.

The verbose solver verifies the AES-GCM tag. A wrong secret gives a wrong key and the tag verification fails, so successful decryption is a strong correctness check.

2. Acronyms and abbreviations for newcomers

Acronym	Full form	Meaning in this writeup
AES	Advanced Encryption Standard	Block cipher used to encrypt the final flag.
GCM	Galois/Counter Mode	Authenticated-encryption mode used with AES.
AEAD	Authenticated Encryption with Associated Data	Encryption family providing confidentiality and integrity.
GF	Galois Field	Finite field; here, the core algebra lives in GF(2^43).
JSON	JavaScript Object Notation	Format of output.json.

PNG	Portable Network Graphics	Image format used for exported diagrams.
DOCX	Microsoft Word document format	Editable writeup format.
Sage / SageMath	Mathematics software system	Useful for finite fields and matrices.
SHA-256	Secure Hash Algorithm 256-bit	Hash used to derive the AES key.

3. Terminology and module interaction

3.1 Basic terminology before challenge-specific names

Term	Meaning
Finite field	A finite set where addition, multiplication and division by non-zero values remain valid.
Matrix	A rectangular table of field elements.
Invertible matrix	A matrix that can be undone by multiplying with its inverse.
Codeword	Encoded version of a message.
Error vector	Noise added to hide the exact codeword.
Rank-metric code	Error-correcting code where error size is measured by rank.
Gabidulin code	Rank-metric analogue of Reed-Solomon codes, built from Frobenius powers.

3.2 Weaving vocabulary mapping

Weaving name	Meaning in this challenge
GF(2^43)	Finite field used for every vector and matrix element.
pegs	Gabidulin support points.
loom	Public Gabidulin generator matrix.
knot	Public invertible k x k left mask.
shuttle	Public invertible n x n right mask built from a 3D fiber space.
fibers	Small subspace that limits error expansion through shuttle.
frays	Low-rank disturbance added to the codeword.
warp	Public masked generator.
bolt	Public noisy masked received word.
vault	AES-GCM ciphertext of the flag.

3.3 How the solver modules work together

```
parse handout -> rebuild finite field -> undo shuttle mask -> decode Gabidulin word -> invert knot
-> decrypt vault
```

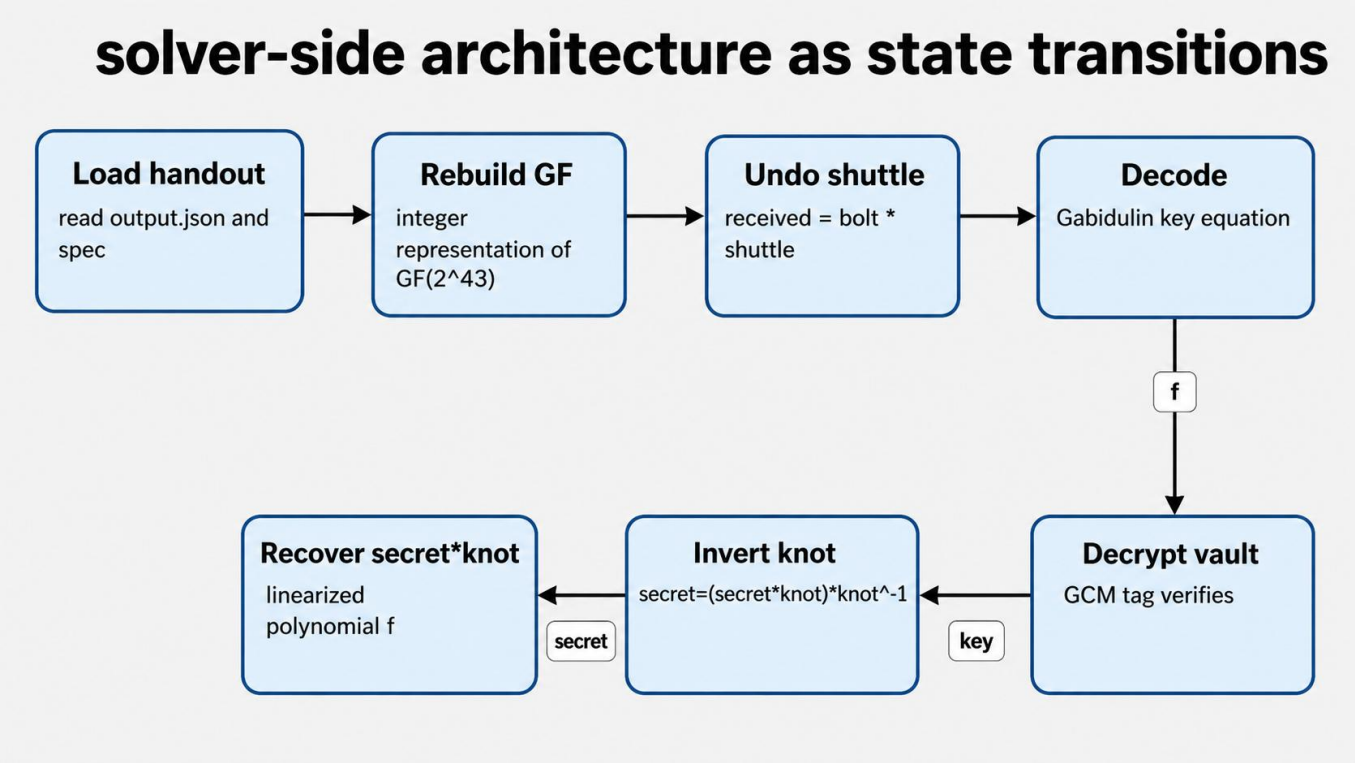


Figure 1 - Solver-side architecture as state transitions.

4. Python solver vs Sage/Docker workflow

Mode	Use when	Command
Pure Python	Final solve	python3 solve_weave_final.py output.json
Pure Python verbose	Proof and debugging	python3 solve_weave_final.py output.json -v
Docker Sage	Reproduce in Sage-compatible environment	docker run --rm -it -v "$PWD:/work" -w /work sagemath/sagemath:latest sage -python solve_weave_final.py output.json -v

5. Reconnaissance and entry-point proof

A clean crypto workflow starts by reading the generator. The decisive lines are the construction of loom, knot, shuttle, warp and bolt.

```
loom    = Matrix(Fqm, K, N, lambda j, i: qpow(pegs[i], j))
knot    = pick_invertible_Fqm(K)
shuttle = pick_shuttle(N, fibers)
warp    = knot * loom * shuttle.inverse()
bolt    = secret * warp + frays
```

Observation	Consequence
knot is public	The left mask can be inverted after decoding.
shuttle is public	The right mask can be cancelled with bolt * shuttle.
pegs are public	The Gabidulin support points are known.

fibers have dimension 3	The post-shuttle error-rank bound is controlled.
AES key depends only on secret	Recovering secret is sufficient to decrypt the vault.

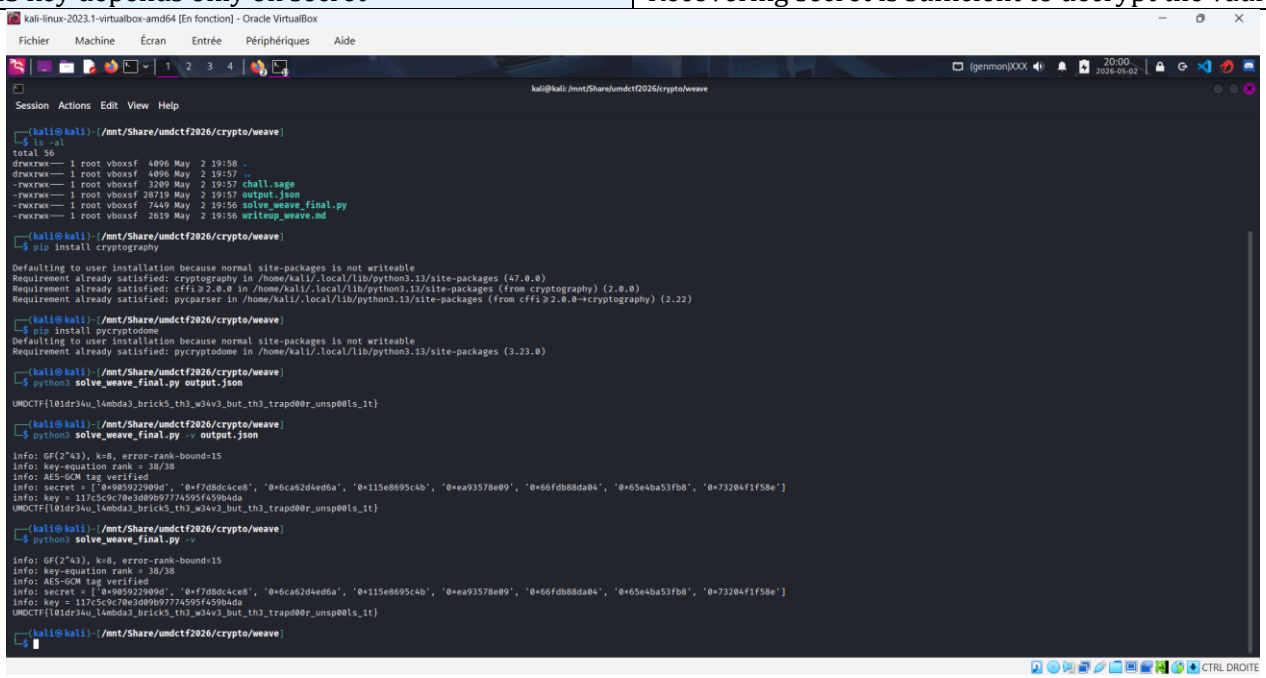


Figure 2 - Terminal proof with local solver output and verbose verification.

6. Mathematical formulation and algebraic unmasking

warp = knot · loom · shuttle^-1

bolt = secret · warp + ε

bolt = secret · knot · loom · shuttle^-1 + ε

bolt · shuttle = secret · knot · loom + ε · shuttle

received = message · loom + e, with message = secret · knot and e = ε · shuttle

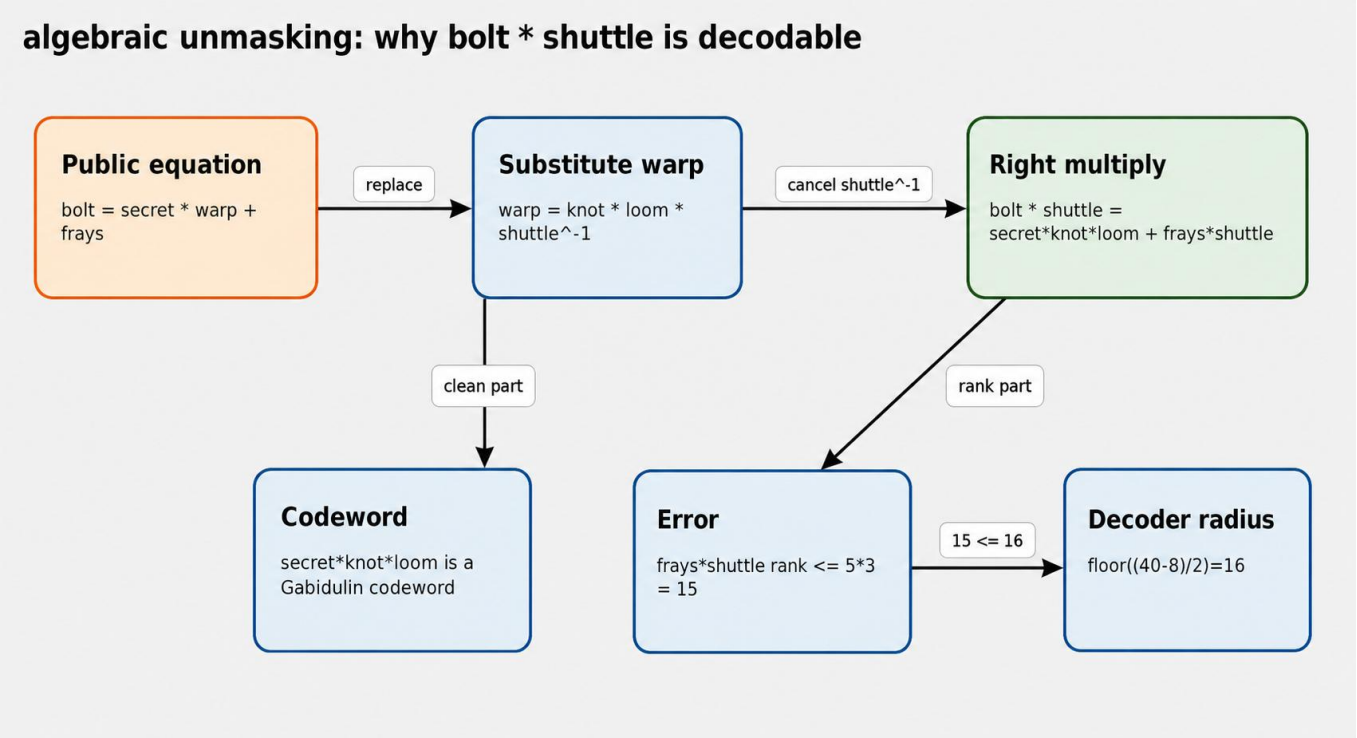


Figure 3 - Algebraic unmasking: composing the public pieces correctly.

7. End-to-end solving flow

7.1 Data-processing state boxes

```
┌──────────────────────────────────────────┐
│ 1. Load public handout                   │
│    chall.sage + output.json              │
└──────────────────────────────────────────┘
                    v
┌──────────────────────────────────────────┐
│ 2. Parse parameters                      │
│    q=2, m=43, n=40, k=8, frays=5         │
└──────────────────────────────────────────┘
                    v
┌──────────────────────────────────────────┐
│ 3. Rebuild GF(2^43)                      │
│    modulus = x^43 + x^21 + x^3 + x + 1   │
└──────────────────────────────────────────┘
                    v
┌──────────────────────────────────────────┐
│ 4. Lift public vectors/matrices          │
│    pegs, knot, shuttle, bolt             │
└──────────────────────────────────────────┘
                    v
┌──────────────────────────────────────────┐
│ 5. Cancel right mask                     │
│    received = bolt · shuttle             │
└──────────────────────────────────────────┘
                    v
┌──────────────────────────────────────────┐
│ 6. Prove error bound                     │
│    rank(e) <= frays × dim(fibers) = 15   │
└──────────────────────────────────────────┘
                    v
```

```
┌──────────────────────────────────────────┐
│ 7. Gabidulin decode                      │
│    recover secret · knot                 │
└──────────────────────────────────────────┘
                    v
┌──────────────────────────────────────────┐
│ 8. Remove left mask                      │
│    secret = (secret · knot) · knot^-1    │
└──────────────────────────────────────────┘
                    v
┌──────────────────────────────────────────┐
│ 9. Derive AES-GCM key and decrypt vault  │
│    SHA256(secret_bytes)[:16]             │
└──────────────────────────────────────────┘
                    v
              ┌──────────┐
              │   FLAG   │
              └──────────┘
```

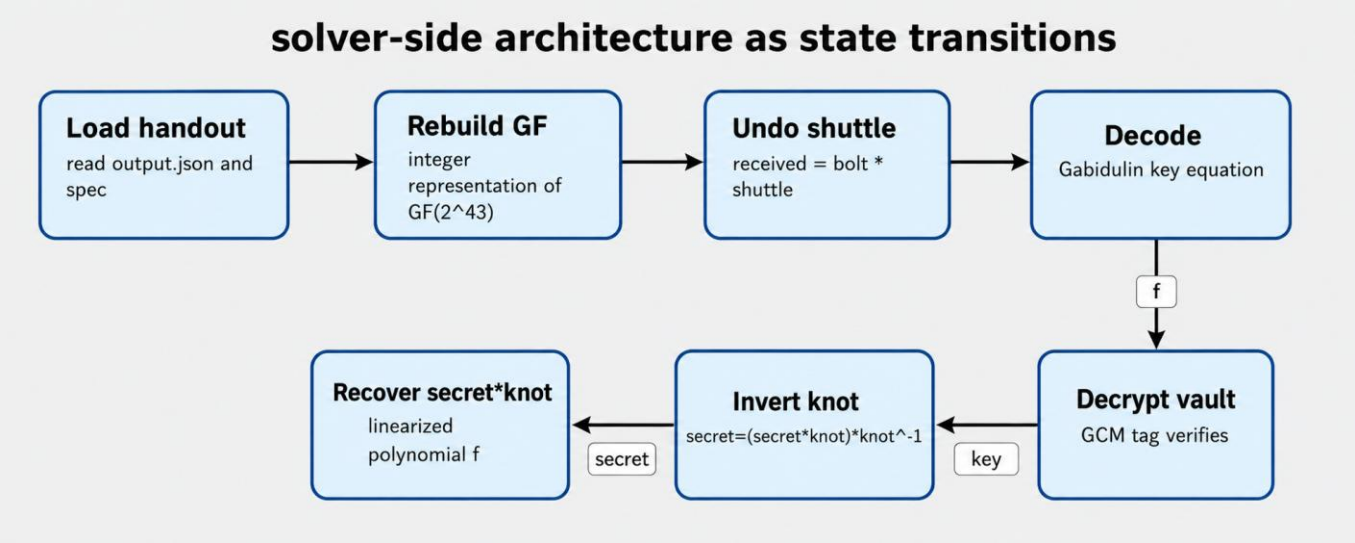


Figure 4 - Data-processing pipeline PNG.

7.2 Challenge-side generation figure

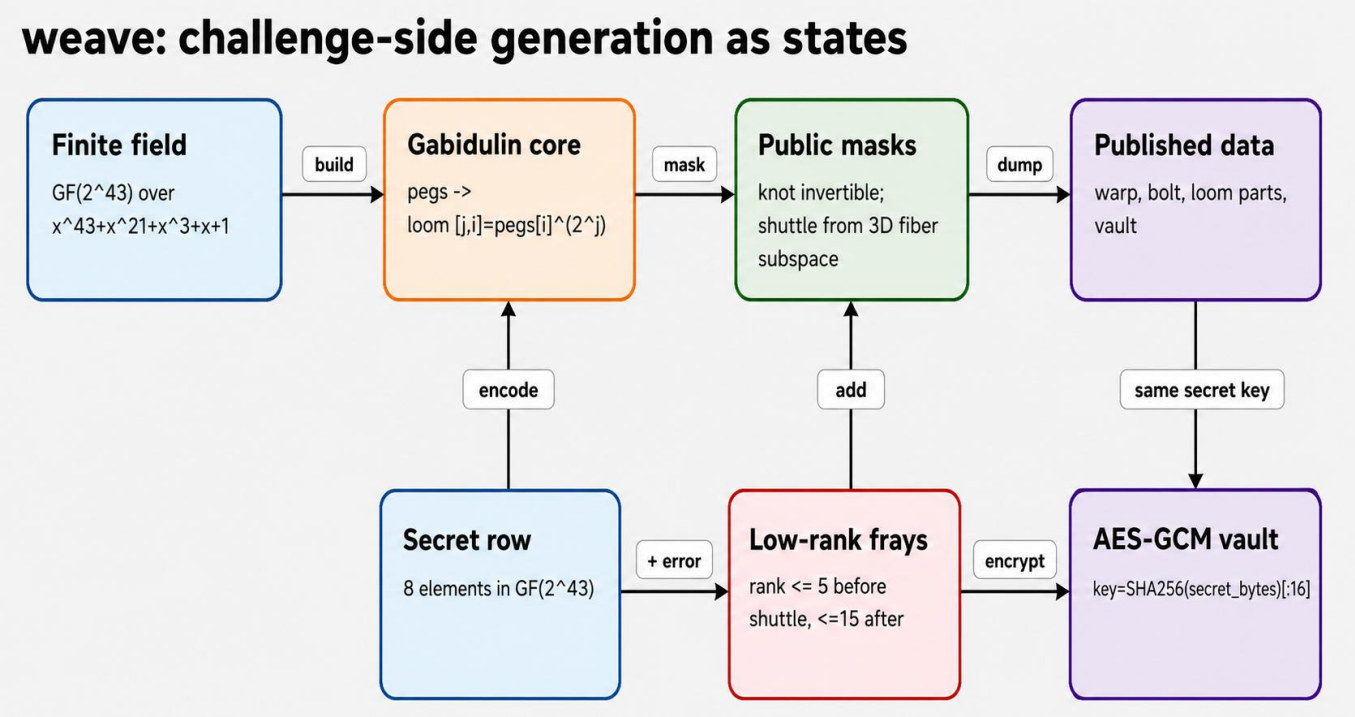


Figure 5 - Challenge-side generation as state transitions.

7.3 Solver state-transition boxes

```
[START]
   |
   v
┌──────────────────────┐
│ Parse public files   │
└──────────┬───────────┘
           v
┌──────────────────────┐
│ Build finite field   │
└──────────┬───────────┘
           v
┌──────────────────────┐
│ Compose public masks │
└──────────┬───────────┘
           v
┌──────────────────────┐
│ Prove decodability   │
└──────────┬───────────┘
           v
┌──────────────────────┐
│ Decode Gabidulin     │
└──────────┬───────────┘
           v
┌──────────────────────┐
│ Untie knot           │
└──────────┬───────────┘
```

```
          v
┌──────────────────────┐
│ Verify AES-GCM tag   │
└──────────┬───────────┘
           v
        [FLAG]
```

8. Reading the flow in detail

The central transformation is bolt * shuttle = secret * knot * loom + frays * shuttle. This cancels the public shuttle^-1 contained in warp and exposes a Gabidulin codeword corrupted by a low-rank error.
The rank bound follows from the construction: rank(frays * shuttle) <= 5 * 3 = 15. For n=40 and k=8, the Gabidulin correction radius is floor((40-8)/2)=16. Therefore t=15 is within the unique-decoding radius.
Remarks: n this challenge, **loom** and **warp** are two matrices over the finite field GF(2^43).
loom is the clean, structured Gabidulin generator matrix. It is built from the public values pegs:

loom[j, i] = pegs[i]^(2^j)

So loom is the matrix that turns a message into a Gabidulin codeword. It is the “real” coding-theory structure.
warp is the masked public version of loom. It is built as:

warp = knot * loom * shuttle^-1

So warp hides the clean Gabidulin structure behind two public invertible masks:

knot = left-side mask
shuttle = right-side mask

In the encryption relation, the challenge uses warp:

bolt = secret * warp + frays

But because shuttle is public, we can multiply by it:
bolt * shuttle = secret * knot * loom + frays * shuttle
That is the key trick: warp looks masked, but after multiplying by shuttle, the clean loom structure comes back.

Very short version:

loom = clean Gabidulin generator
warp = masked version of loom used to produce bolt.

9. Decoding proof as transition boxes

```
┌──────────────────────────────────────────┐
│ Start from public received word          │
│ received = bolt · shuttle                │
└──────────────────────────────────────────┘
                    v
┌──────────────────────────────────────────┐
│ Rewrite with the public construction     │
│ received = (secret·knot)·loom + e        │
└──────────────────────────────────────────┘
                    v
┌──────────────────────────────────────────┐
│ Bound transformed error                  │
│ rank(e) <= 5 × 3 = 15                    │
└──────────────────────────────────────────┘
                    v
┌──────────────────────────────────────────┐
│ Compute Gabidulin decoding radius        │
│ floor((n-k)/2) = floor((40-8)/2) = 16    │
└──────────────────────────────────────────┘
                    v
          ┌──────────────────┐
          │ Is 15 <= 16 ?    │
          └──────────────────┘
                    | yes
                    v
┌──────────────────────────────────────────┐
│ Unique decoding applies                  │
│ decoder returns secret · knot            │
└──────────────────────────────────────────┘
                    v
┌──────────────────────────────────────────┐
│ Multiply by knot^-1                      │
│ recover secret                           │
└──────────────────────────────────────────┘
```

the intended trapdoor: all pieces are public, but composition matters

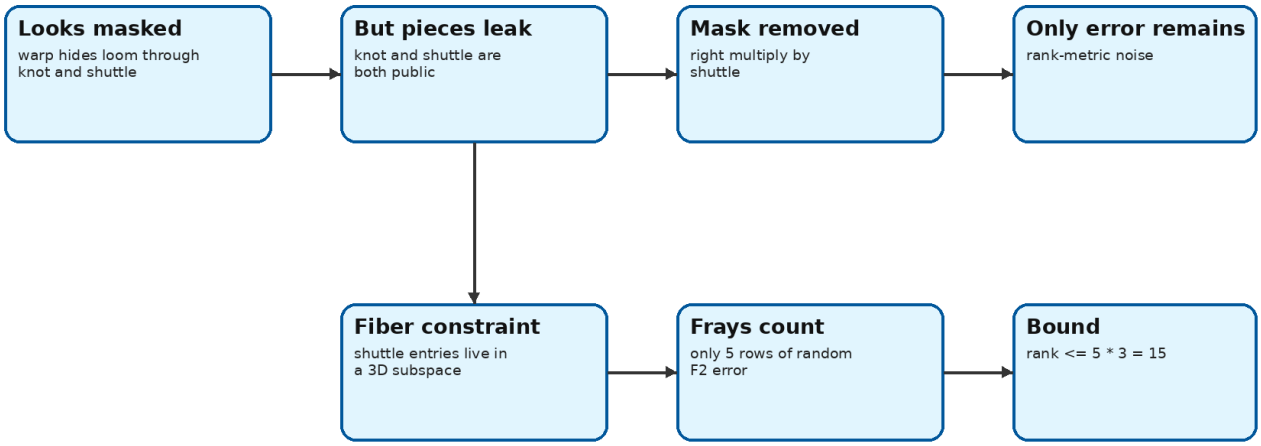


Figure 6 - Low-rank trapdoor and decoding proof.

10. Stack layout equivalent: algebra layout and payload derivation

```
bolt = secret * warp + frays
     = secret * knot * loom * shuttle^-1 + frays

bolt * shuttle = secret * knot * loom + frays * shuttle
               = Gabidulin codeword + rank <= 15 error
```

Derived object	Formula	Purpose
Received word	received = bolt * shuttle	Remove shuttle^-1.
Error bound	t = frays * len(fibers) = 15	Select decoder degree.
Decoding radius	floor((n-k)/2) = 16	Prove t=15 is correctable.
Masked secret	secret_times_knot	Decoder output.
Final secret	secret_times_knot * knot^-1	AES key material.

11. What the low-rank trapdoor means here

The masks are not secret. The only remaining protection is the rank-metric error, and the published fiber restriction keeps that error too small. This is why decoding works once shuttle has been cancelled.

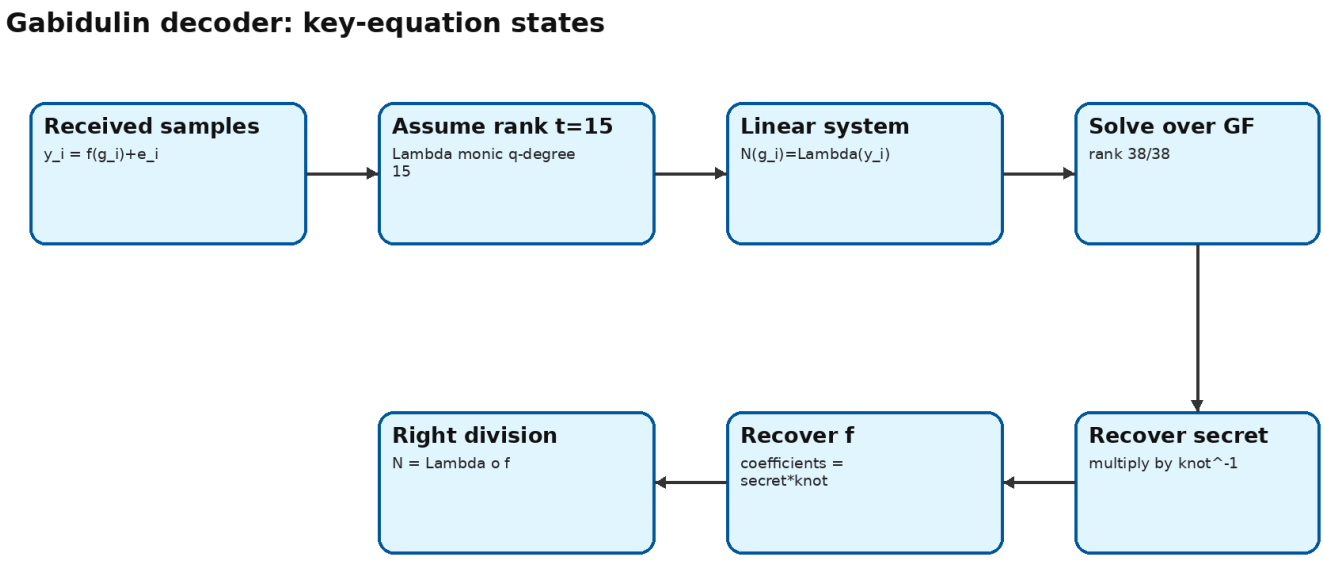


Figure 7 - Gabidulin decoder key-equation states.

12. Compiling, setup and solver launch options

Strictly speaking, there is no native compilation step because the solver is Python. The useful equivalent is syntax-checking, dependency setup, and choosing the execution mode.

Option type	Purpose	Command	When to use it
Syntax check / pseudo-compile	Verify that the solver parses	python3 -m py_compile solve_weave_final.py	Before running, especially after editing the solver.
Minimal dependency install	Install AES dependency	python3 -m pip install pycryptodome	If Crypto.Cipher.AES is missing.
Optional crypto backend	Install alternative crypto package	python3 -m pip install cryptography	Useful in environments preferring cryptography, but not required by the PyCryptodome solver.
Virtual environment	Create a clean Python environment	python3 -m venv .venv && . .venv/bin/activate && pip install pycryptodome	Recommended for reproducible local runs.
Direct solve	Run the final pure-Python solver	python3 solve_weave_final.py output.json	Preferred final solve command.
Verbose solve	Run with detailed logs	python3 solve_weave_final.py output.json -v	Best for screenshots, proof, and debugging.
Docker Sage verification	Run inside a Sage-compatible image	docker run --rm -it -v "$PWD:/work" -w /work sagemath/sagemath:latest sage -python solve_weave_final.py output.json -v	When checking behavior against Sage finite-field semantics.
Challenge inspection	Confirm public parameters	sed -n "1,220p" chall.sage && jq ".spec" output.json	Confirms q, m, n, k, frays, and modulus.

13. Minimal local proof without Sage

```
python3 -m pip install pycryptodome
python3 solve_weave_final.py output.json -v

Expected result:
AES-GCM tag verified


UMDCTF{l01dr34u_l4mbda3_brick5_th3_w34v3_but_th3_trapd00r_unsp00ls_1t}
```

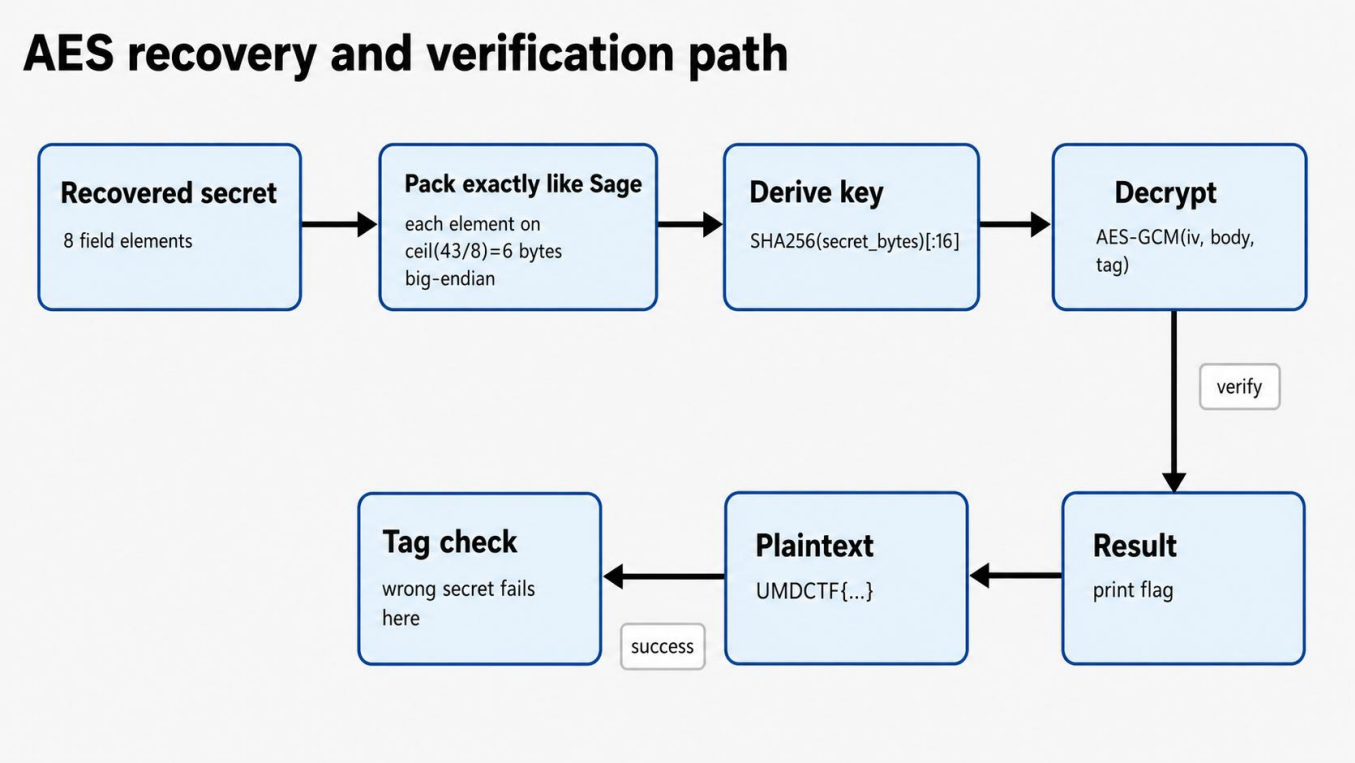


Figure 8 - AES recovery and verification path.

14. Other ways to discover the payload

The discovery path is reproducible: identify that warp is only a masked version of loom, multiply bolt by shuttle to cancel the public right-side mask, inspect pick_shuttle to find the 3D fiber subspace, inspect pick_frays to find FRAYS = 5, compute the rank bound 5 × 3 = 15, then compare it with the decoding radius 16; this proves that the noisy value is not random anymore, but a valid Gabidulin decoding instance inside the correctable range.

14.1 Why this is the best path for this challenge

This path attacks the weak layer. AES-GCM is not broken, and the field secret is far too large to brute-force. The efficient route is structural rank-metric decoding after composing the public matrices in the intended order, because this directly transforms the public data into the exact mathematical problem that the decoder can solve.

15. Code excerpts, pitfalls and trial-and-error

```
received = row_times_matrix(bolt, shuttle, field)
error_rank_bound = frays * len(fibers)

# N(g_i) = Lambda(y_i)
row = [field.qpow(pegs[i], j) for j in range(k + t)]
row += [field.qpow(received[i], j) for j in range(t)]

secret_bytes = b"".join(x.to_bytes((m + 7) // 8, "big") for x in secret)
key = hashlib.sha256(secret_bytes).digest()[:16]
```

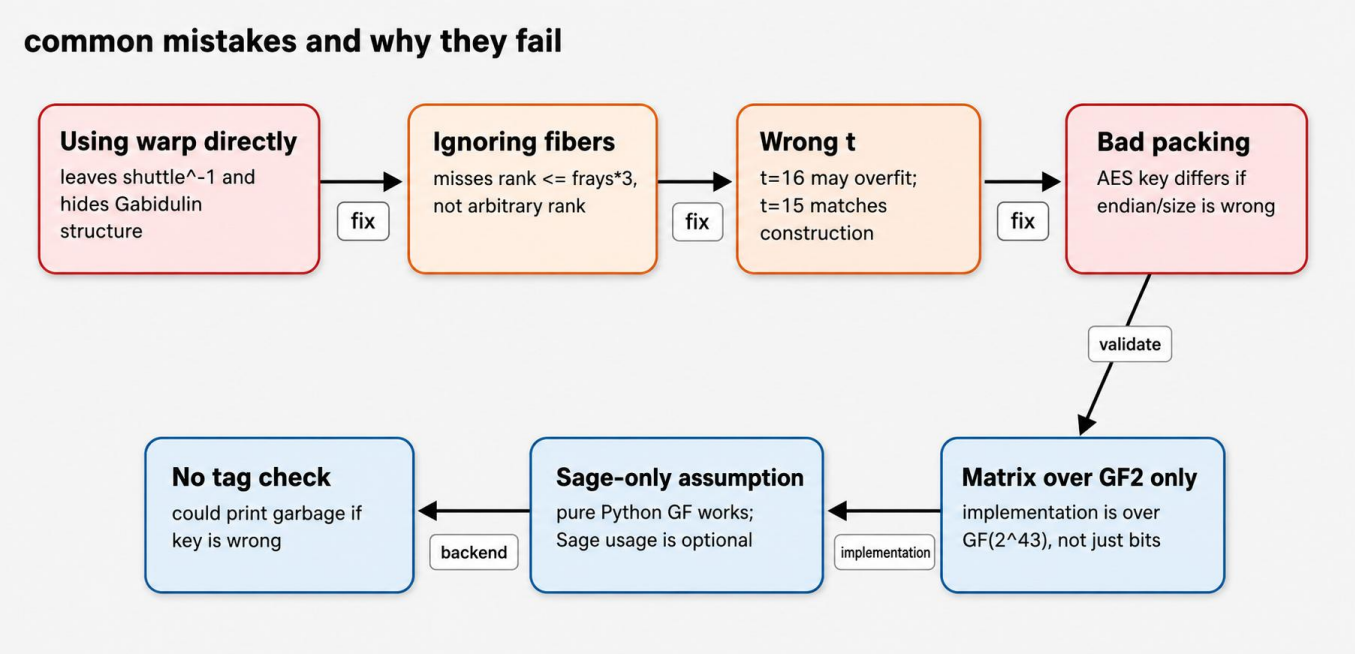


Figure 9 - Common mistakes and why they fail.

Mistake	Why it fails	Better approach
Treating values as integers	All arithmetic must happen in GF(2^43).	Rebuild the field first.
Attacking AES-GCM directly	AES-GCM is not the weak layer.	Recover the secret from the code layer.
Ignoring shuttle	The received word remains masked.	Compute received = bolt * shuttle.
Skipping the rank proof	The decoder looks unjustified.	Explicitly prove 15 <= 16.
Removing knot too early	Decoder returns secret * knot, not secret.	Decode first, then multiply by knot^-1.
Re-implementing AES-GCM manually	It adds risk for no benefit.	Use a standard library.

16. Adapted meme comments

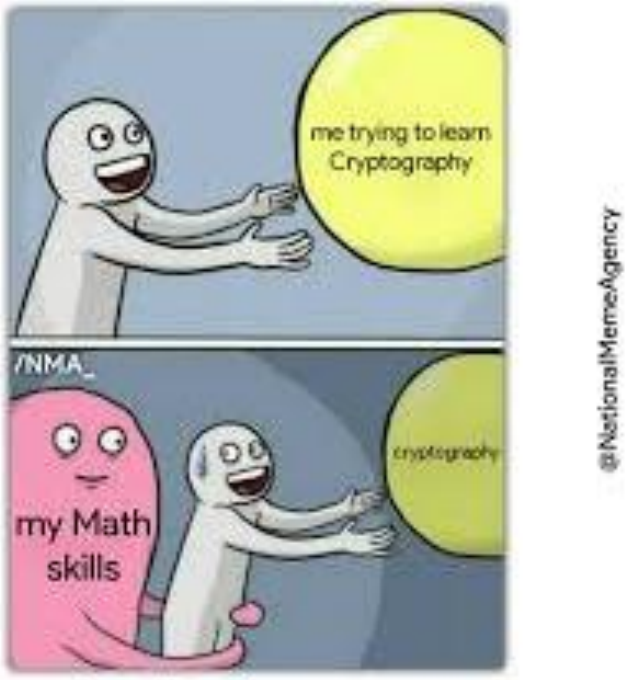
Name	Writeup interpretation
loom	The real Gabidulin generator.
knot	A left-side mask tied around the message.
shuttle	A right-side mask that can be cancelled because it is public.
fibers	The small subspace that limits error expansion.
frays	The low-rank noise.
warp	The public masked generator.
bolt	The received noisy word.
vault	The AES-GCM wrapping of the flag.

```
unshuttle the bolt -> decode the frays -> untie the knot -> open the vault
```

Meme 1 - Cross-platform meme

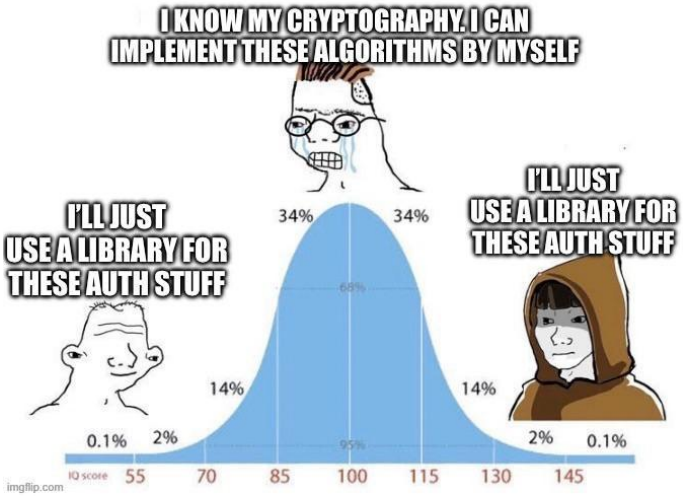
We do not need a universal attack against every rank-metric scheme. This instance already gives the specific public structure we need.

Meme 2 - Math skills vs cryptography meme



The bottleneck is finite-field and rank-metric reasoning, not Python syntax.

Meme 3 - Use a library for authentication stuff



Meaning: handle the custom algebra carefully, but use a vetted library for AES-GCM.

17. Complementary layered solving map

The solving layers are: JSON parsing, finite-field reconstruction, public-mask cancellation, Gabidulin decoding, knot inversion, Sage-compatible secret packing, SHA-256 key derivation, and AES-GCM tag verification.

18. Solver variant A - pure Python final solver

This is the recommended solver. It has no Sage dependency and supports both normal and verbose runs.

```
python3 solve_weave_final.py output.json
python3 solve_weave_final.py output.json -v
```

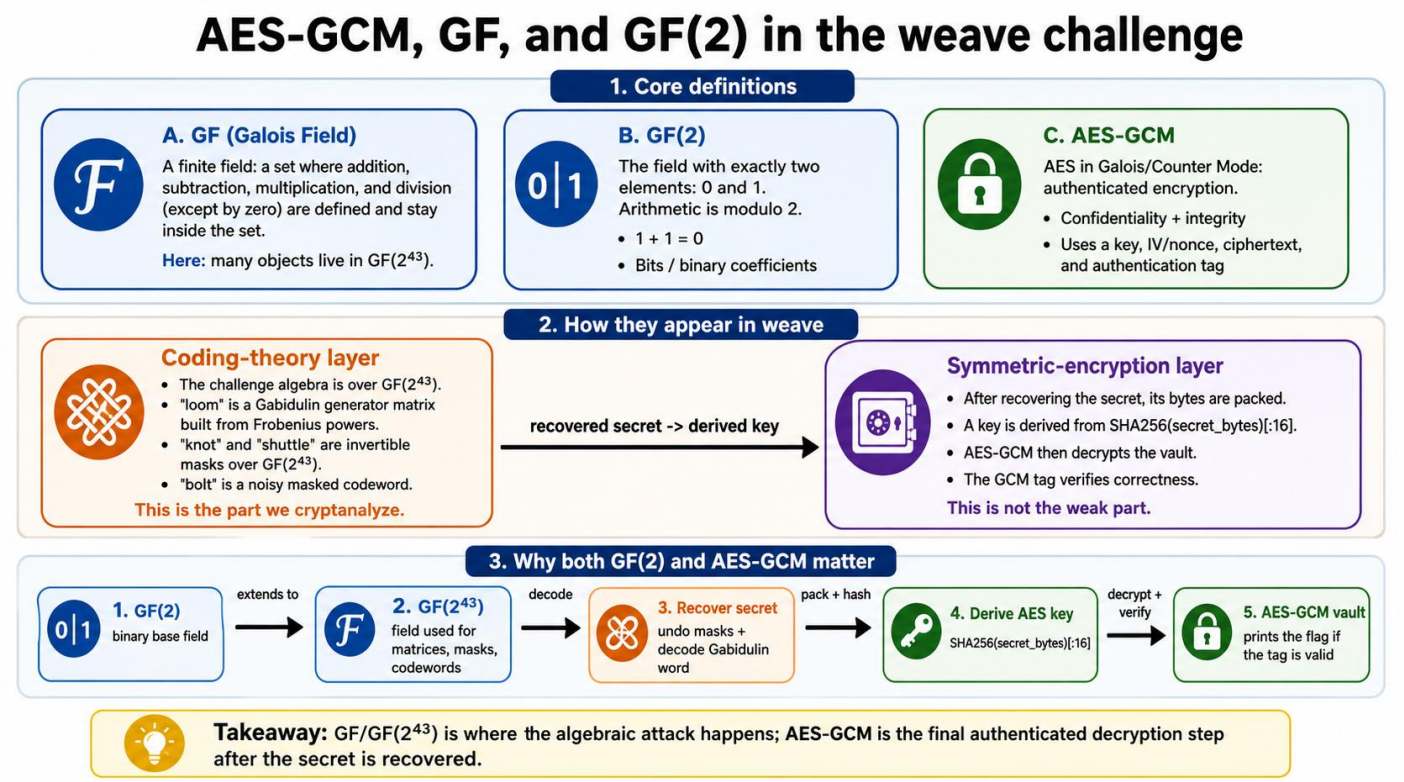
19. Solver variant B - Sage/Docker verification path

There is no second exploit path needed, but Sage is useful as a verification environment.

```
docker run --rm -it \
  -v "$PWD:/work" \
  -w /work \
  sagemath/sagemath:latest \
  sage -python solve_weave_final.py output.json -v
```

20. Synthesis

The decisive relation is bolt * shuttle = secret * knot * loom + frays * shuttle. The right side is a Gabidulin codeword plus an error of rank at most 15. This equation shows that, after removing the shuttle mask, the right-hand side becomes a Gabidulin codeword plus a controlled error. The error has rank at most 15.
The Gabidulin code parameters are n = 40 and k = 8, so the code can correct up to floor((40 - 8) / 2) = 16 rank errors. Since the error rank is at most 15, decoding is possible.
The decoder recovers secret * knot. Then, because knot is public and invertible, multiplying by knot^-1 gives the real secret. This recovered secret is then used to derive the AES-GCM key, which verifies and decrypts the flag.



21. Particular error-correction

21.1. Error measure code

A **Gabidulin code** is an error-correcting code built over a finite field extension, typically GF(qm)GF(q^m)GF(qm), where errors are measured by **rank** instead of by the number of wrong positions.

In simpler words:

- A normal code like Reed–Solomon often measures errors by **Hamming distance**: "how many symbols changed?"
- A Gabidulin code measures errors by **rank distance**: "how many independent directions over the base field were introduced by the error?"

That is why Gabidulin codes are often described as the **rank-metric analogue of Reed–Solomon codes**. SageMath's documentation describes them this way and defines them as evaluation codes of degree-restricted skew/linearized polynomials.

Where does the notion come from?

The name comes from **Ernst M. Gabidulin**, who introduced this family in the paper **"Theory of Codes with Maximum Rank Distance"**, published in 1985 in *Problems of Information Transmission*. The paper studies codes over GF(qN)GF(q^N)GF(qN), introduces the rank metric, and describes maximum rank distance codes with encoding and decoding algorithms.

Historically, rank-metric codes were also studied earlier by **Delsarte** in 1978, but the specific Gabidulin-code construction is associated with Gabidulin's 1985 work.

What does a Gabidulin generator look like?

A Gabidulin code uses support elements g1,…,gng_1, \dots, g_ng1,…,gn in GF(qm)GF(q^m)GF(qm), chosen to be linearly independent over GF(q)GF(q)GF(q). Its generator matrix has a Frobenius-power structure:

G =
[g1 g2 ... gn]
[g1^q g2^q ... gn^q]
[g1^(q^2) g2^(q^2) ... gn^(q^2)]
[... ... ...]

In the weave challenge, q = 2, so the powers are:
g, g^2, g^(2^2), g^(2^3), ...
That is why this line is the key clue:

loom = Matrix(Fqm, K, N, lambda j, i: pegs[i] ** (2 ** j))

It has exactly the Gabidulin shape.

How is this used in weave?
In the challenge, loom is the Gabidulin generator matrix. The challenge then hides it with public masks:
warp = knot * loom * shuttle^-1
and publishes:
bolt = secret * warp + frays
At first, bolt looks like a noisy masked object. But since shuttle is public, we multiply by it:
bolt * shuttle
= secret * knot * loom + frays * shuttle
Now the right-hand side is:
Gabidulin codeword + low-rank error
That is exactly what Gabidulin decoding is designed to handle.

Why does decoding work here?

The error after unmasking has rank at most:
5 * 3 = 15
The code has:
n = 40
k = 8
so its correction radius is:
floor((n - k) / 2) = floor(32 / 2) = 16
Since:
15 <= 16
the Gabidulin decoder can recover:
secret * knot
Then, because knot is public and invertible:
secret = (secret * knot) * knot^-1
So in this challenge, Gabidulin codes are not just background theory. They are the **exact reason** why the masked value becomes decodable once bolt is multiplied by shuttle.

21.2. Quick-reference table

Object	Formula	Role
received	bolt * shuttle	remove right mask
error bound	5 * 3 = 15	prove decodability
radius	floor((40-8)/2)=16	unique decoding
message	secret * knot	decoder output
secret	message * knot^-1	AES key material

22. Solver variant A - pure Python final solver

This section embeds the complete Python solver directly into the writeup, following the same style as the pwn writeup: the reader can understand the workflow, then copy the full script without needing to open a separate file. *The standalone file is still kept as solve_weave_final.py for execution.*

22.1 Launch and environment options

Option	Purpose	Command	Use when
Syntax check	Pseudo-compilation for Python	`python3 -m py_compile solve_weave_final.py`	Before running after edits
Dependency install	AES backend	`python3 -m pip install pycryptodome`	When Crypto.Cipher.AES is missing
Alternative AES backend	Fallback backend	`python3 -m pip install cryptography`	If pycryptodome is unavailable
Normal solve	Final direct run	`python3 solve_weave_final.py output.json`	Clean final solution
Verbose solve	Proof/debug output	`python3 solve_weave_final.py output.json -v`	Writeup evidence and debugging
Docker Sage parity	Sage-compatible container	`docker run --rm -it -v "$PWD:/work" -w /work sagemath/sagemath:latest sage -python solve_weave_final.py output.json -v`	Compare with the challenge Sage environment

22.2 Solver function map

The full script is easier to read if each group of functions is mapped to the algebraic proof first. The table below follows the actual execution order.

Solver part	Function(s)	Role in the proof
Finite-field engine	GF2m.mul / pow / inv / qpow / qroot	Rebuilds arithmetic in GF(2^43), including Frobenius powers and inverses.
Handout parsing	json.load / modulus_from_coefficients	Reads output.json and reconstructs the field modulus from the public coefficients.
Matrix operations	row_times_matrix / invert_matrix	Computes received = bolt * shuttle and later removes knot with knot^-1.
Linear algebra	solve_linear_system_fqm	Solves the Gabidulin key equation over GF(2^43).
Rank-metric decoder	decode_gabidulin	Recovers secret * knot from the noisy Gabidulin word.
Key derivation	derive_key	Packs the recovered secret exactly like the challenge and derives SHA256(secret_bytes)[:16].
Vault opening	aes_gcm_decrypt	Uses AES-GCM to verify the tag and decrypt the final flag.
Entry point	main	Connects the whole pipeline: parse, unmask, decode, unmask knot, derive key, decrypt.

22.3 Why this solver is deterministic

The solver does not brute-force the flag or the AES key. It rewrites the public relation into received = bolt * shuttle, where the right-hand side is a Gabidulin codeword plus an error whose rank is bounded by 15.

Because the code parameters give a decoding radius of 16, the decoder can recover secret * knot. Since knot is public and invertible, one final matrix multiplication gives the real secret.

AES-GCM is used only after that point. The authentication tag verifies the recovered key; if the algebraic recovery were wrong, decrypt_and_verify would fail.

22.4 Complete solver code - solve_weave_final.py

The code is embedded verbatim from the standalone solver file. It is intentionally pure Python except for the AES-GCM backend, which uses pycryptodome and falls back to cryptography if needed.

```
#!/usr/bin/env python3
"""
UMDCTF crypto/weave solver.

Usage:
  python3 solve_weave.py output.json
  python3 solve_weave.py output.json -v

Dependencies:
  - pycryptodome:  pip install pycryptodome
    or cryptography: pip install cryptography
"""

import argparse
```

```
import hashlib
import json


class GF2m:
    def __init__(self, m: int, modulus: int):
        self.m = m
        self.modulus = modulus
        self.mask = (1 << m) - 1

    def mul(self, a: int, b: int) -> int:
        out = 0
        while b:
            if b & 1:
                out ^= a
            b >>= 1
            a <<= 1
            if a & (1 << self.m):
                a ^= self.modulus
        return out & self.mask

    def pow(self, a: int, e: int) -> int:
        out = 1
        while e:
            if e & 1:
                out = self.mul(out, a)
            a = self.mul(a, a)
            e >>= 1
        return out

    def inv(self, a: int) -> int:
        if a == 0:
            raise ZeroDivisionError("zero has no inverse")
        return self.pow(a, (1 << self.m) - 2)

    def qpow(self, a: int, j: int) -> int:
        # Frobenius power: a^(2^j)
        j %= self.m
        for _ in range(j):
            a = self.mul(a, a)
        return a

    def qroot(self, a: int, j: int) -> int:
        # Inverse of a -> a^(2^j) in GF(2^m)
        return self.qpow(a, -j)

def modulus_from_coefficients(coeffs: list[int]) -> int:
    value = 0
    for i, c in enumerate(coeffs):
        if c:
            value |= 1 << i
    return value


def row_times_matrix(row: list[int], matrix: list[list[int]], field: GF2m) -> list[int]:
    cols = len(matrix[0])
    out = [0] * cols
    for i, value in enumerate(row):
        if value:
            for j in range(cols):
                out[j] ^= field.mul(value, matrix[i][j])
    return out


def invert_matrix(matrix: list[list[int]], field: GF2m) -> list[list[int]]:
    n = len(matrix)
    work = [
        matrix[i][:] + [1 if i == j else 0 for j in range(n)]
        for i in range(n)
    ]

    for col in range(n):
        pivot = next((r for r in range(col, n) if work[r][col]), None)
        if pivot is None:
            raise ValueError("singular matrix")

        work[col], work[pivot] = work[pivot], work[col]
        inv_pivot = field.inv(work[col][col])
        work[col] = [field.mul(x, inv_pivot) for x in work[col]]

        for r in range(n):
            if r != col and work[r][col]:
                factor = work[r][col]
                work[r] = [
                    work[r][j] ^ field.mul(factor, work[col][j])
                    for j in range(2 * n)
                ]

    return [row[n:] for row in work]

def solve_linear_system_fqm(
    matrix: list[list[int]],
    rhs: list[int],
    field: GF2m,
) -> tuple[list[int], int]:
    rows = len(matrix)
    cols = len(matrix[0])
    work = [matrix[i][:] + [rhs[i]] for i in range(rows)]
    pivots = []
    rank = 0
```

```
    for col in range(cols):
        pivot = next((r for r in range(rank, rows) if work[r][col]), None)
        if pivot is None:
            continue

        work[rank], work[pivot] = work[pivot], work[rank]
        inv_pivot = field.inv(work[rank][col])
        work[rank] = [field.mul(x, inv_pivot) for x in work[rank]]

        for r in range(rows):
            if r != rank and work[r][col]:
                factor = work[r][col]
                work[r] = [
                    work[r][j] ^ field.mul(factor, work[rank][j])
                    for j in range(cols + 1)
                ]

        pivots.append(col)
        rank += 1

    for r in range(rank, rows):
        if all(work[r][c] == 0 for c in range(cols)) and work[r][cols]:
            raise ValueError("inconsistent system")

    solution = [0] * cols
    for r, col in enumerate(pivots):
        solution[col] = work[r][cols]

    return solution, rank


def decode_gabidulin(
    received: list[int],
    pegs: list[int],
    k: int,
    error_rank_bound: int,
    field: GF2m,
    verbose: bool = False,
) -> list[int]:
    """
    Decodes y_i = f(pegs_i) + e_i with rank(e) <= error_rank_bound.

    It solves the linearized key equation:
      N(g_i) = Lambda(y_i)
    with deg_q(N) < k+t and monic deg_q(Lambda)=t.
    """
    t = error_rank_bound
    n = len(pegs)
    n_unknowns = k + t
    cols = n unknowns + t

    matrix = []
    rhs = []
    for i in range(n):
        row = [field.qpow(pegs[i], j) for j in range(n_unknowns)]
        row += [field.qpow(received[i], j) for j in range(t)]
        matrix.append(row)
        rhs.append(field.qpow(received[i], t))

    solution, rank = solve_linear_system_fqm(matrix, rhs, field)
    if verbose:
        print(f"info: key-equation rank = {rank}/{cols}")

    N = solution[:n_unknowns]
    Lambda = solution[n unknowns:] + [1]  # monic term

    # Right division of linearized polynomials: N = Lambda o f.
    f = [0] * k
    for degree in range(k + t - 1, t - 1, -1):
        acc = N[degree]
        for a in range(t):
            b = degree - a
            if 0 <= b < k and f[b]:
                acc ^= field.mul(Lambda[a], field.qpow(f[b], a))
        f[degree - t] = field.qroot(acc, t)

    return f

def aes_gcm_decrypt(key: bytes, iv: bytes, body: bytes, tag: bytes) -> bytes:
    try:
        from Crypto.Cipher import AES
        cipher = AES.new(key, AES.MODE_GCM, nonce=iv)
        return cipher.decrypt_and_verify(body, tag)
    except ModuleNotFoundError:
        from cryptography.hazmat.primitives.ciphers.aead import AESGCM
        return AESGCM(key).decrypt(iv, body + tag, None)


def derive_key(secret: list[int], m: int) -> bytes:
    element_size = (m + 7) // 8
    secret_bytes = b"".join(x.to_bytes(element_size, "big") for x in secret)
    return hashlib.sha256(secret_bytes).digest()[:16]

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("output_json", nargs="?", default="output.json")
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()

    handout = json.load(open(args.output_json, "r", encoding="utf-8"))
    spec = handout["spec"]
```

```
    m = spec["m"]
    k = spec["k"]
    frays = spec["frays"]
    field = GF2m(m, modulus_from_coefficients(spec["modulus"]))

    bolt = handout["bolt"]
    loom = handout["loom"]
    pegs = loom["pegs"]
    knot = loom["knot"]
    shuttle = loom["shuttle"]
    fibers = loom["fibers"]

    # Undo the public shuttle mask:
    #   bolt * shuttle = (secret * knot) * loom + frays * shuttle
    # The error rank is at most frays * len(fibers) = 15.
    received = row_times_matrix(bolt, shuttle, field)
    error rank bound = frays * len(fibers)

    if args.verbose:
        print(f"info: GF(2^{m}), k={k}, error-rank-bound={error_rank_bound}")

    secret_times_knot = decode_gabidulin(
        received,
        pegs,
        k,
        error rank bound,
        field,
        verbose=args.verbose,
    )

    secret = row_times_matrix(secret_times_knot, invert_matrix(knot, field), field)
    key = derive key(secret, m)

    vault = handout["vault"]
    flag = aes_gcm_decrypt(
        key,
        bytes.fromhex(vault["iv"]),
        bytes.fromhex(vault["body"]),
        bytes.fromhex(vault["tag"]),
    )

    if args.verbose:
        print("info: AES-GCM tag verified")
        print("info: secret =", [hex(x) for x in secret])
        print("info: key =", key.hex())

    print(flag.decode())

if   name   == "  main  ":
    main()
```

22.5. Practical copy/paste note

- Save the code block above as solve_weave_final.py next to output.json.
- Install pycryptodome if needed, then run: python3 solve_weave_final.py output.json -v.
- A successful run prints the flag and, in verbose mode, confirms that the AES-GCM tag verified.

•

23. Larger conclusion

The solution follows the construction instead of attacking the wrong layer. AES-GCM is not weak here; it only acts as the final authenticated encryption layer around the flag. Trying to break AES-GCM directly, guess the key, or brute-force the secret would be unrealistic and would ignore the real structure exposed by the challenge.
The weak point is the public composition of a Gabidulin instance whose error rank remains below the decoding radius. In other words, the challenge gives enough public algebraic material to transform the published value bolt into a standard decoding problem. Multiplying by shuttle cancels the right-side mask, and the remaining expression becomes:

bolt * shuttle = secret * knot * loom + frays * shuttle

This is the decisive relation. The term secret * knot * loom is a valid Gabidulin codeword, while frays * shuttle is a controlled low-rank error. Because this error has rank at most 15, and the Gabidulin code can correct up to 16 rank errors, the decoder can recover secret * knot.

The last mask, knot, is also public and invertible. Once decoding gives secret * knot, multiplying by knot^-1 recovers the real secret. From there, the AES-GCM layer becomes straightforward: the solver packs the secret bytes exactly as the challenge does, derives SHA256(secret_bytes)[:16], and decrypts the vault. The GCM tag then confirms that the recovered secret is correct. . Once the algebra is written down, the final solver is deterministic and small.

Final flag: UMDCTF{l01dr34u_l4mbda3_brick5_th3_w34v3_but_th3_trapd00r_unsp00ls_1t}

So the key lesson is methodological: the challenge is not solved by attacking the strongest-looking cryptographic primitive. It is solved by identifying the layer where the construction leaks structure, rewriting the equations, and using the intended algebraic weakness. Once that algebra is written down, the final solver is deterministic, compact, and reliable.